

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 2
по дисциплине «Анализ алгоритмов»

Тема Вычисление процессорного времени реализаций различных алгоритмов расчета редакционных растояний.

Студент Одинцов Е.В.

Группа ИУ7-63БВ

Преподаватели Волкова Л.Л.

Москва, 2025

Содержание

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**

1.1 Определение расстояния Левенштейна 5

1.2 Алгоритмы вычисления расстояния Левенштейна 5

2 Технологическая часть **6**

2.1 Структура проекта 6

2.2 Реализация алгоритмов расчета редакционных расстояний . . . 6

3 Исследовательская часть **17**

3.1 Методология замера времени работы реализаций алгоритмов расчета редакционных расстояний 17

3.2 Результаты замеров времени 17

3.3 Анализ результатов 19

ЗАКЛЮЧЕНИЕ **20**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **21**

ВВЕДЕНИЕ

Расстояние Левенштейна — метрика, которая определяет, насколько две строки отличаются друг от друга. Оно определяется как минимальное количество элементарных операций (вставки, удаления, замены), необходимых для преобразования одной строки в другую. Расстояние Левенштейна имеет широкий спектр применений в различных областях:

— обработка естественного языка и исправление опечаток;

— распознавание речи;

— биоинформатика (анализ последовательностей ДНК);

— сравнение текстов и поиск плагиата;

— рекомендательные системы.

В данной работе проводится сравнительный анализ четырех различных реализаций алгоритма вычисления расстояния Левенштейна: базовый итеративный алгоритм, оптимизированный по памяти итеративный алгоритм, рекурсивный алгоритм и рекурсивный алгоритм с мемоизацией. Цель исследования — на практике сравнить эффективность этих реализаций и определить оптимальный вариант для различных условий.

1 Аналитическая часть

1.1 Определение расстояния Левенштейна

Расстояние Левенштейна между двумя строками a и b можно определить рекурсивно. Обозначим через $lev(i, j)$ минимальное количество операций (вставка, удаление, замена), необходимых для преобразования первых i символов строки a в первые j символов строки b. Предполагается, что индексация символов начинается с 1. Тогда:

$$lev(i,j) = \begin{cases} j, & \text{если } i = 0 \\ i, & \text{если } j = 0 \\ lev(i-1, j-1), & \text{если } a_i = b_j \\ 1 + \min \begin{cases} lev(i-1, j), \\ lev(i, j-1), \\ lev(i-1, j-1) \end{cases}, & \text{иначе} \end{cases} \tag{1.1}$$

1.2 Алгоритмы вычисления расстояния Левенштейна

В работе рассматриваются четыре реализации алгоритма:

1) **Стандартный итеративный алгоритм:** использует матрицу размера $(m + 1) \times (n + 1)$ для динамического программирования. Сложность по времени $O(m \times n)$, по памяти $O(m \times n)$.

2) **Оптимизированный по памяти алгоритм:** использует только два вектора длины $n+1$ для вычислений. Сложность по времени остается $O(m \times n)$, но сложность по памяти сокращается до $O(n)$.

3) **Рекурсивный алгоритм:** прямая реализация рекурсивной формулы. Сложность по времени $O(3^{m+n})$ из-за повторных вычислений, по памяти $O(m + n)$ для стека вызовов.

4) **Рекурсивный алгоритм с мемоизацией:** рекурсивная реализация с сохранением результатов промежуточных вычислений. Сложность по времени $O(m \times n)$, по памяти $O(m \times n)$.

2 Технологическая часть

2.1 Структура проекта

Для реализации алгоритмов был использован язык программирования C++. Проект содержит следующие ключевые файлы:

— `Levenshtein.h` — заголовочный файл с объявлениями класса и методов

— `Levenshtein.cpp` — реализация методов

— `string1based.h` и `string1based.cpp` — вспомогательный класс для работы со строками с индексацией от 1

— `main.cpp` — основная программа для тестирования и сравнения алгоритмов

2.2 Реализация алгоритмов расчета редакционных расстояний

Реализация алгоритмов расчета редакционных расстояний Левенштейна представлена на листингах 2.1 – 2.2.

Листинг 2.1 — Заголовочный файл Levenshtein.h

```
#ifndef LEVENSHTEIN_LEVENSHTEIN_H
#define LEVENSHTEIN_LEVENSHTEIN_H

#include "string1based.h"
#include <vector>
#include <iostream>
#include <algorithm>

class Levenshtein {
  private:

    int cost(const String1Based &s1, const String1Based &s2, int i, int j);

    int min(std::vector<std::vector<int>> &D, const String1Based &s1, const String1Based &s2, int i, int j);
```

```
  public:

  void print_matrix(const std::vector<std::vector<int>> &D);

  static int distance(const String1Based &s1, const
 String1Based &s2);

  static int optimizedDistance(const String1Based &s1, const
 String1Based &s2);

  static int recursiveDistance(const String1Based &s1, const
 String1Based &s2, int i, int j);
  static int recursiveDistance(const String1Based &s1, const
 String1Based &s2);

  static int memoizedRecursiveDistance(
  const String1Based &s1,
  const String1Based &s2,
  int i, int j,
  std::vector<std::vector<int>> &memo);
  static int memoizedRecursiveDistance(const String1Based &s1,
 const String1Based &s2);
};

#endif //LEVENSHTEIN_LEVENSHTEIN_H
```

Листинг 2.2 — Реализация методов в Levenshtein.cpp

```
#include "Levenshtein.h"

int Levenshtein::cost(const String1Based &s1, const
 String1Based &s2, int i, int j) {
  return (s1[i] == s2[j]) ? 0 : 1;
}

int Levenshtein::min(std::vector<std::vector<int>> &D, const
 String1Based &s1, const String1Based &s2, int i, int j) {
  return std::min(
  {
    D[i - 1][j] + 1,
```

```
      D[i][j - 1] + 1,
      D[i - 1][j - 1] + cost(s1, s2, i, j)
    });
  }

  void Levenshtein::print_matrix(const std::vector<std::vector<
   int>> &D) {
    for (const auto &row: D) {
      for (int elem: row) {
        std::cout << elem << " ";
      }
      std::cout << std::endl;
    }
  }

  // Стандартный итеративный алгоритм
  int Levenshtein::distance(const String1Based &s1, const
   String1Based &s2) {
    int len1 = s1.length();
    int len2 = s2.length();

    std::vector<std::vector<int>> D(len1 + 1, std::vector<int>(
   len2 + 1, 0));

    for (int i = 0; i <= len1; i++) {
      D[i][0] = i;
    }

    for (int j = 0; j <= len2; j++) {
      D[0][j] = j;
    }

    for (int i = 1; i <= len1; i++) {
      for (int j = 1; j <= len2; j++) {
        D[i][j] = min(D, s1, s2, i, j);
      }
    }

    return D[len1][len2];
  }
```

```
48
49  // Оптимизированный по памяти алгоритм
50  int Levenshtein::optimizedDistance(const String1Based &s1,
     const String1Based &s2) {
51    int len1 = s1.length();
52    int len2 = s2.length();
53
54    std::vector<int> prev_row(len2 + 1);
55    std::vector<int> curr_row(len2 + 1);
56
57    for (int j = 0; j <= len2; j++) {
58      prev_row[j] = j;
59    }
60
61    for (int i = 1; i <= len1; i++) {
62      curr_row[0] = i;
63      for (int j = 1; j <= len2; j++) {
64        curr_row[j] = std::min(
65        {
66          prev_row[j] + 1,
67          curr_row[j - 1] + 1,
68          prev_row[j - 1] + cost(s1, s2, i, j)
69        });
70      }
71      prev_row.swap(curr_row);
72    }
73
74    return prev_row[len2];
75  }
76
77  // Рекурсивный алгоритм
78  int Levenshtein::recursiveDistance(const String1Based &s1,
     const String1Based &s2, int i, int j) {
79    if (i == 0) {
80      return j;
81    }
82    if (j == 0) {
83      return i;
84    }
85
```

```
    return std::min(
    {
      recursiveDistance(s1, s2, i - 1, j) + 1,
      recursiveDistance(s1, s2, i, j - 1) + 1,
      recursiveDistance(s1, s2, i - 1, j - 1) + cost(s1, s2, i, j
 )
    });
}

int Levenshtein::recursiveDistance(const String1Based &s1,
 const String1Based &s2) {
    return recursiveDistance(s1, s2, s1.length(), s2.length());
}

// Рекурсивный алгоритм с мемоизацией
int Levenshtein::memoizedRecursiveDistance(
const String1Based &s1,
const String1Based &s2,
int i, int j,
std::vector<std::vector<int>> &memo) {

    if (memo[i][j] != -1) {
      return memo[i][j];
    }
    if (i == 0) {
      return j;
    }
    if (j == 0) {
      return i;
    }

    memo[i][j] = std::min(
    {
      memoizedRecursiveDistance(s1, s2, i - 1, j, memo) + 1,
      memoizedRecursiveDistance(s1, s2, i, j - 1, memo) + 1,
      memoizedRecursiveDistance(s1, s2, i - 1, j - 1, memo) +
 cost(s1, s2, i, j)
    });

    return memo[i][j];
```

```
}

int Levenshtein::memoizedRecursiveDistance(const String1Based &s1, const String1Based &s2) {
  int len1 = s1.length();
  int len2 = s2.length();
  std::vector<std::vector<int>> memo(len1 + 1, std::vector<int>(len2 + 1, -1));

  for (int i = 0; i <= len1; i++) {
    memo[i][0] = i;
  }

  for (int j = 0; j <= len2; j++) {
    memo[0][j] = j;
  }

  return memoizedRecursiveDistance(s1, s2, s1.length(), s2.length(), memo);
}
```

Реализация вспомогательного класса для работы со строками с индексацией от 1 приведена в листингах 2.3 – 2.4

Листинг 2.3 — Заголовочный файл класса String1Based

```
#ifndef LEVENSTEIN_STRING1BASED_H
#define LEVENSTEIN_STRING1BASED_H


#include <string>
#include <stdexcept>
#include <utility>

class String1Based {
  std::string data;
  public:
  explicit String1Based(std::string s = "");

  char &operator[](size_t pos);

```

```
  const char &operator[](size_t pos) const;

  size_t length() const;

  const std::string &str() const;

  void append(char c);

  void append(const std::string &s);
};

#endif //LEVENSTEIN_STRING1BASED_H
```

Листинг 2.4 — Файл реализации класса String1Based

```
#include "string1based.h"

String1Based::String1Based(std::string s) : data(std::move(s)) {}

char &String1Based::operator[](size_t pos) {
  if (pos < 1 || pos > data.length()) {
    throw std::out_of_range("String index out of range (1-based)"
   );
  }
  return data[pos - 1];
}

const char &String1Based::operator[](size_t pos) const {
  if (pos < 1 || pos > data.length()) {
    throw std::out_of_range("String index out of range (1-based)"
   );
  }
  return data[pos - 1];
}

size_t String1Based::length() const {
  return data.length();
}

const std::string &String1Based::str() const {
```

```
  return data;
}

void String1Based::append(char c) {
  data += c;
}

void String1Based::append(const std::string &s) {
  data += s;
}
```

Реализация основной программы приведена в листинге 2.5

Листинг 2.5 — Программа main.cpp

```
#include "string1based.h"
#include "Levenshtein.h"
#include <windows.h>
#include <fstream>
#include <vector>

LARGE_INTEGER getPerformanceCounter() {
  LARGE_INTEGER count;
  QueryPerformanceCounter(&count);
  return count;
}

double getTimeDiff(LARGE_INTEGER start, LARGE_INTEGER end,
 LARGE_INTEGER frequency) {
  return static_cast<double>(end.QuadPart - start.QuadPart) /
 frequency.QuadPart;
}

double measureLevenshteinTime(
int (*levenshteinFunction)(const String1Based &, const
 String1Based &),
const String1Based &s1,
const String1Based &s2) {

  LARGE_INTEGER frequency;
  QueryPerformanceFrequency(&frequency);

```

```
    double totalTime = 0;
    int runs = 5;
    for (int i = 0; i < runs; ++i) {
      LARGE_INTEGER start = getPerformanceCounter();
      int result = levenshteinFunction(s1, s2);
      LARGE_INTEGER end = getPerformanceCounter();
      totalTime += getTimeDiff(start, end, frequency);
    }
    return totalTime / runs;
}

int main() {
    String1Based s1("GIBRALTAR");
    String1Based s2("LABRADOR");

    std::vector<std::pair<String1Based, String1Based>> testCases;
    testCases.push_back({String1Based("G"), String1Based("L")});
    testCases.push_back({String1Based("GI"), String1Based("LA")});
    testCases.push_back({String1Based("GIB"), String1Based("LAB")});
    testCases.push_back({String1Based("GIBR"), String1Based("LABR")});
    testCases.push_back({String1Based("GIBRA"), String1Based("LABRA")});
    testCases.push_back({String1Based("GIBRAL"), String1Based("LABRAD")});
    testCases.push_back({String1Based("GIBRALT"), String1Based("LABRADO")});
    testCases.push_back({String1Based("GIBRALTA"), String1Based("LABRADOR")});
    testCases.push_back({String1Based("GIBRALTAR"), String1Based("LABRADOR_")});

    std::ofstream file("levenshtein_times.csv");
    file << "StringLength,Method,Time\n";
    std::cout << "Verifying correctness..." << std::endl;
    int result1 = Levenshtein::distance(s1, s2);
    int result2 = Levenshtein::optimizedDistance(s1, s2);
    int result3 = Levenshtein::recursiveDistance(s1, s2);
```

```
    int result4 = Levenshtein::memoizedRecursiveDistance(s1, s2);

    std::cout << "Standard: " << result1 << std::endl;
    std::cout << "Optimized: " << result2 << std::endl;
    std::cout << "Recursive: " << result3 << std::endl;
    std::cout << "Memoized Recursive: " << result4 << std::endl;

    if (result1 == result2 && result2 == result3 && result3 == result4) {
        std::cout << "All methods produce the same result: " << result1 << std::endl;
    } else {
        std::cout << "WARNING: Methods produce different results!" << std::endl;
    }

    std::cout << "\nRunning benchmarks..." << std::endl;

    for (const auto &test: testCases) {
        int length = test.first.length();
        std::cout << "Testing strings of length " << length << std::endl;

        double time1 = measureLevenshteinTime(Levenshtein::distance, test.first, test.second);
        file << length << ",Standard," << time1 << "\n";

        double time2 = measureLevenshteinTime(Levenshtein::optimizedDistance, test.first, test.second);
        file << length << ",Optimized," << time2 << "\n";

        double time3 = measureLevenshteinTime(Levenshtein::recursiveDistance, test.first, test.second);
        file << length << ",Recursive," << time3 << "\n";

        double time4 = measureLevenshteinTime(Levenshtein::memoizedRecursiveDistance, test.first, test.second);
        file << length << ",MemoizedRecursive," << time4 << "\n";
    }

```

```
    file.close();
    std::cout << "Benchmark complete. Results saved to
 levenshtein_times.csv" << std::endl;

    return 0;
}
```

3 Исследовательская часть

3.1 Методология замера времени работы реализаций алгоритмов расчета редакционных расстояний

Для сравнения эффективности различных реализаций алгоритма расстояния Левенштейна было проведено тестирование на строках различной длины. Для каждой пары строк было измерено среднее время выполнения каждого из четырех алгоритмов. Измерения проводились с использованием высокоточного таймера Windows (функция `QueryPerformanceCounter` [6]).

Для корректности эксперимента выполнялось следующее.

— Для замера времени вводились пары строк одинаковой длины, эта длин варьировалась.

— Каждый реализация алгоритм запускалась для каждого входного значения по 5 раз, результат замера процессорного времени усреднялся.

— Замеры выполнялись на строках длиной от 1 до 7 символов.

— Проверялась корректность результатов работы реализаций всех алгоритмов.

3.2 Результаты замеров времени

Результаты замера процесорного времени работы реализаций алгоритмов поиска расстояния Левенштейна представлены на рис. 3.1 – 3.2.

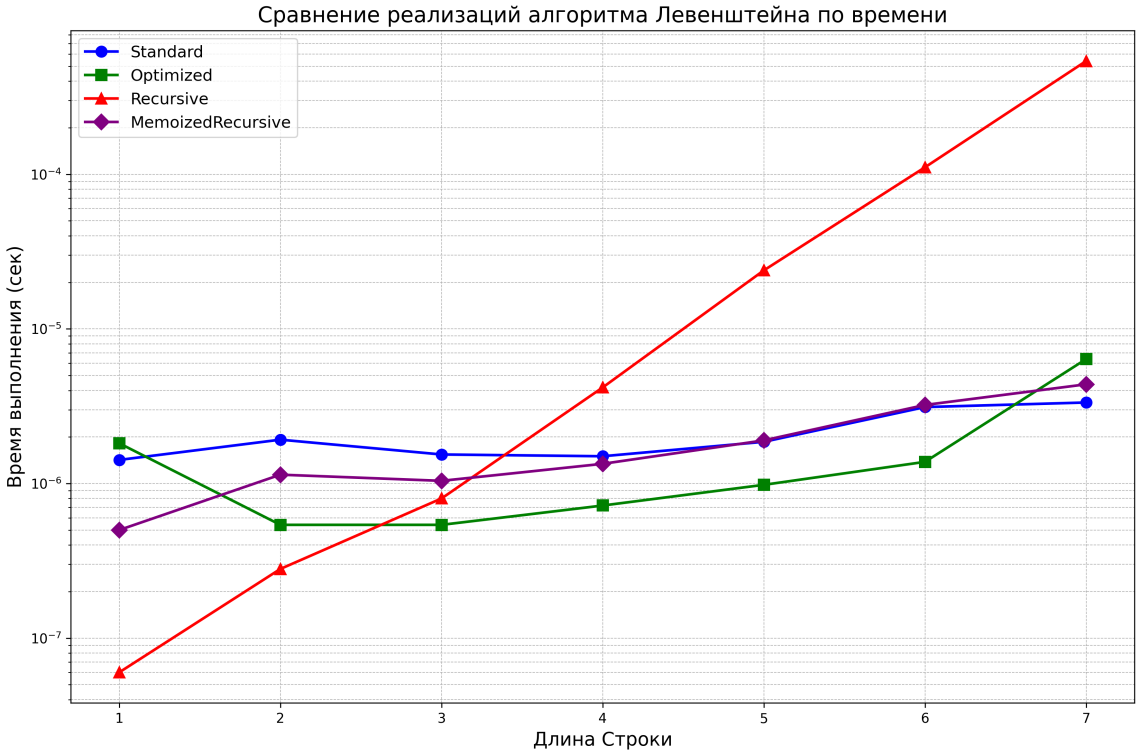


Рисунок 3.1 — Сравнение времени выполнения реализаций алгоритмов Левенштейна (логарифмическая шкала)

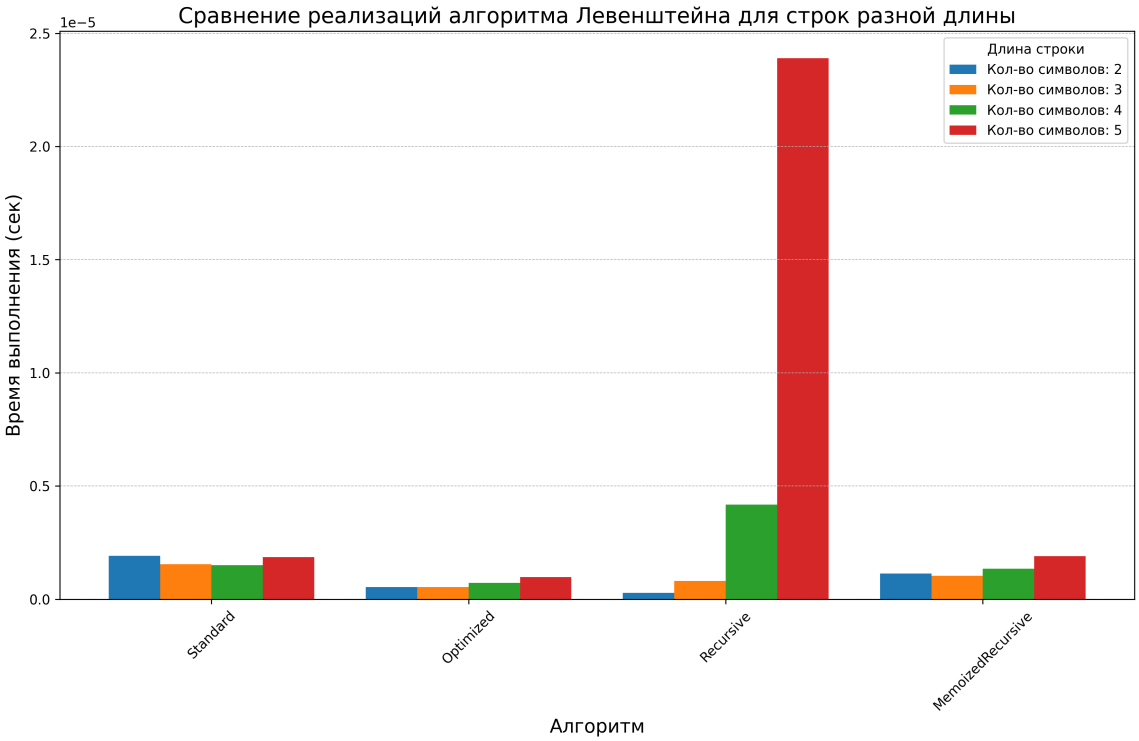


Рисунок 3.2 — Сравнение времени выполнения реализаций алгоритмов Левенштейна для строк разной длины

3.3 Анализ результатов

На основе проведенных измерений можно сделать следующие выводы:

1) **Стандартный итеративный алгоритм (Standard)** демонстрирует стабильно хорошую производительность со сложностью $O(m \times n)$, что подтверждается линейным ростом времени выполнения в зависимости от длины строк.

2) **Оптимизированный по памяти алгоритм (Optimized)** показывает производительность, сравнимую со стандартным итеративным алгоритмом, но с существенно меньшим потреблением памяти. Для строк большой длины эта реализация может быть предпочтительной из-за экономии памяти.

3) **Рекурсивный алгоритм без оптимизации (Recursive)** показывает экспоненциальный рост времени выполнения с увеличением длины строк. Уже для строк длиной 8-9 символов время выполнения становится неприемлемым для практического использования.

4) **Рекурсивный алгоритм с мемоизацией (Memoized Recursive)** демонстрирует производительность, близкую к итеративным алгоритмам, но с небольшими накладными расходами на рекурсивные вызовы. Это подтверждает, что мемоизация эффективно решает проблему повторных вычислений в рекурсивном алгоритме.

Для коротких строк (до 3-4 символов) все алгоритмы, кроме чисто рекурсивного, показывают сравнимую производительность. Для строк средней и большой длины оптимизированный по памяти алгоритм и стандартный итеративный алгоритм демонстрируют наилучшую производительность.

ЗАКЛЮЧЕНИЕ

В ходе работы были реализованы и исследованы четыре различных алгоритма вычисления расстояния Левенштейна. Каждый из алгоритмов имеет свои преимущества и недостатки, которые делают их более или менее подходящими для различных сценариев использования.

Основные выводы исследования:

1) Итеративный алгоритм на основе матрицы динамического программирования обеспечивает хорошую производительность, но требует $O(m \times n)$ памяти.
2) Оптимизированный по памяти алгоритм сохраняет временную эффективность итеративного алгоритма, но требует только $O(n)$ памяти, что делает его предпочтительным для длинных строк.
3) Простой рекурсивный алгоритм хорошо иллюстрирует математическое определение расстояния Левенштейна, но неэффективен на практике из-за экспоненциальной сложности.
4) Рекурсивный алгоритм с мемоизацией также эффективен, но имеет небольшие накладные расходы на рекурсивные вызовы.

На практике рекомендуется использовать либо стандартный итеративный алгоритм для строк средней длины, либо оптимизированный по памяти алгоритм для очень длинных строк, когда экономия памяти становится критичной.

Данное исследование наглядно демонстрирует важность выбора правильного алгоритма в зависимости от конкретных условий задачи и подтверждает эффективность методов динамического программирования для решения задач на строках.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ульянов М.В. Ресурсно-эффективные компьютерные алгоритмы. — М.: ФИЗМАТЛИТ, 2008. — 304 с.

2. Джеффри Макконнелл. Анализ алгоритмов. Активный обучающий подход. — М.: Техносфера, 2009. — 416 с.

3. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — М.: Вильямс, 2015. — 1328 с.

4. Седжвик Р. Алгоритмы на C++. — М.: Вильямс, 2019. — 1056 с.

5. Кнут Д.Э. Искусство программирования. Том 3. Сортировка и поиск. — М.: Диалектика-Вильямс, 2019. — 832 с.

6. Microsoft. QueryPerformanceCounter function [эл. ресурс]. Режим доступа: `https://learn.microsoft.com/ru-ru/windows/win32/api/profileapi/nf-profileapi-queryperformancecounter` (дата обращения: 22.03.2025).